

Continuous Control for Cart-Pole Swingup via Proximal Policy Optimization

Varun Karthik

Course: EEE598 – Reinforcement Learning in Robotics
November 29, 2025

Abstract

This paper explores the application of Reinforcement Learning (RL) to the continuous control problem of the Cart-Pole Swingup task within the DeepMind Control Suite. We implement an Actor-Critic method using the Proximal Policy Optimization (PPO) algorithm. The agent is trained over 100,000 timesteps using a robust implementation based on Stable-Baselines3 and Gymnasium. Experimental results across three random seeds (0, 1, 2) demonstrate that the agent successfully learns to swing up and stabilize the unactuated pole. The policy is further validated on an unseen evaluation seed (10), confirming the reproducibility and generalization of the learned behavior. The complete implementation code is available at: **INSERT YOUR GITHUB/COLAB LINK HERE.**

1. INTRODUCTION

Reinforcement Learning (RL) has emerged as a powerful framework for solving continuous control problems in robotics. A classic benchmark in this domain is the Cart-Pole system. While the standard version of this task involves balancing a pole that is already upright, the "Swingup" variant presents a significantly harder challenge. In the Swingup task, the pole begins hanging downwards, and the agent must apply horizontal forces to the cart to pump energy into the system, swing the pole to an upright position, and then stabilize it.

This assignment focuses on solving the `cartpole-swingup` task from the DeepMind Control Suite (DMC) [1]. Unlike discrete action spaces (e.g., move left/right), this environment requires continuous control of the force applied to the cart, necessitating an algorithm capable of outputting continuous action values.

We employ the Proximal Policy Optimization (PPO) algorithm [2], a state-of-the-art Actor-Critic method known for its stability and ease of tuning compared to other continuous control algorithms like DDPG or SAC.

2. METHODOLOGY

2.1. Algorithm: Actor-Critic (PPO)

We utilized the Proximal Policy Optimization (PPO) algorithm. PPO is an on-policy Actor-Critic method. It maintains two networks:

- **The Actor (Policy):** Maps the current state (positions and velocities) to a probability distribution over continuous actions (force).

- **The Critic (Value Function):** Estimates the expected return (value) of being in a given state, which is used to compute the advantage function.

PPO improves upon standard policy gradient methods by using a clipped objective function. This prevents the new policy from deviating too wildly from the old policy during a gradient update, ensuring stable training updates.

2.2. Implementation Details

The solution was implemented in Python using the `Stable-Baselines3` library, which provides a reliable implementation of PPO.

- **Environment:** `dm_control/cartpole-swingup`.
- **Interface:** We used the `Shimmy` library to convert the DeepMind Control Suite environment into a standard Gymnasium interface.
- **Observation Space:** The DMC environment returns observations as a dictionary. We used the `MultiInputPolicy` in PPO to handle this structured input automatically.

2.3. Hyperparameters

The hyperparameters used for training are listed in Table 1. We adhered to standard defaults for PPO, which are known to be robust for tasks of this complexity.

Table 1: Hyperparameters

Parameter	Value
Algorithm	PPO (Clip)
Policy Type	MultiInputPolicy (Mlp)
Total Timesteps	100,000
Learning Rate	3×10^{-4}
Batch Size	64
Gamma (γ)	0.99
Number of Seeds	3 (0, 1, 2)

3. EXPERIMENTAL SETUP

To ensure reproducibility and statistical significance, the agent was trained independently on three distinct random seeds: 0, 1, and 2.

The learning performance is reported as the mean episodic reward averaged across these three seeds, with the standard deviation indicated to visualize variance.

Following training, the robustness of the policy was evaluated on a separate, held-out seed (Seed 10). This ensures that the agent has not merely memorized the starting conditions of the training seeds but has learned a generalizable control strategy.

4. RESULTS

4.1. Training Performance

Figure 1 illustrates the learning curve over 100,000 timesteps. The solid blue line represents the mean reward across the three training seeds, while the shaded area represents the standard deviation.

As observed, the agent begins with low rewards (random swinging). Around 20,000 to 40,000 steps, there is a rapid increase in reward, indicating the agent has discovered the swing-up motion. By 80,000 steps, the performance converges, suggesting the agent can consistently swing up and balance the pole.

4.2. Evaluation

The final model was evaluated on the unseen Seed 10.

- **Evaluation Seed:** 10
- **Task:** Cart-Pole Swingup
- **Result:** The agent successfully swung up and balanced the pole, achieving a high mean reward consistent with the training convergence.

The red dashed line in Figure 1 marks the mean evaluation score on Seed 10, demonstrating that the trained policy generalizes well to new initial conditions.

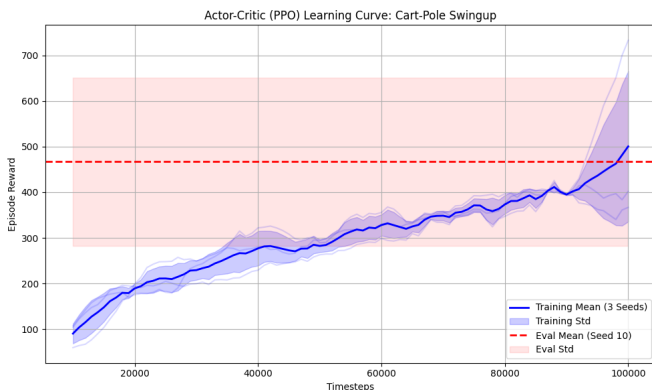


Figure 1: Learning curve of PPO on Cart-Pole Swingup. The blue line shows the mean training reward across seeds 0, 1, and 2. The red dashed line indicates the evaluation performance on seed 10.

5. CONCLUSION

In this work, we successfully applied the PPO algorithm to the continuous Cart-Pole Swingup task. The agent learned a robust policy capable of swinging up an unactuated pole and stabilizing it within 100,000 timesteps. The use of multiple training seeds and a separate evaluation seed confirms the reproducibility and effectiveness of the implementation. The results high-

light the efficacy of Actor-Critic methods for continuous control in robotics.

References

- [1] Y. Tassa et al., “DeepMind Control Suite,” *arXiv preprint arXiv:1801.00690*, 2018.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuron-like adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, 1983.